We use Llama-2-70B for our judge, but you could use GPT4 or any other LLM you prefer (though we cannot guarantee good results!). Our code leverages the Databricks Foundational Model API, which we also used in *Chapter 3* when creating the embeddings of *ArXiv* article text chunks. As a reminder, the Databricks Foundation Model APIs provide direct access to state-of-the-art open models from a serving endpoint, allowing you to incorporate high-quality generative AI models into your application without the need to maintain your own model deployment. We call the Llama-2-70b endpoint in *Figure 7.49*.

```
from mlflow.deployments import get_deploy_client
deploy_client = get_deploy_client("databricks")
endpoint_name = "databricks-llama-2-70b-chat"
#Let's query our external model endpoint
answer_test = deploy_client.predict(endpoint=endpoint_name, inputs={"messages":
[{"role": "user", "content": "What is GPT?"}]})
answer_test['choices'][0]['message']['content']display(spark.table
('evaluation_dataset'))
```

Figure 7.49 – Testing the Foundation Model endpoint for Llama-2-70b

We build a DataFrame from the `evaluation_table` questions and answers. If you have added many more question/answer examples to this dataset, you may want to downsample the number of questions and speed up the prediction process. We then call our UDF `predict_answer`.

```
df_qa = (spark.read.table('evaluation_table')
                .selectExpr('question_asked as inputs', 'answer_given as targets')
                .where("targets is not null")
                #.sample(fraction=0.005, seed=40) # if your dataset is very big you
                could sample it
                )

df_qa_with_preds = df_qa.withColumn('preds', predict_answer(col('inputs'))).cache()
```

Figure 7.50 – Building a DataFrame with questions and ground truth answers, and adding RAG chatbot answers

Now that we have our DataFrame with the chatbot's responses to each question, we save this to a Delta table as well. We continue to reference the DataFrame throughout the rest of this code, but this way we won't have to generate the chatbot's responses again if we want to query this data in the future.

```
df_qa_with_preds.write.saveAsTable(f"{catalog}.{database_name}.
evaluation_table_preds")
```

Figure 7.51 – Writing our evaluation DataFrame to a new table